From Operation Logs to an Automation Proposal

Final report · Repository: https://github.com/insaneado/iby

Summary

Dataset B contains **173 minutes of back-office work: 664 executions across 15 sessions and 4 operators**, all on 2026-07-01. Recovering that required solving the problem the raw logs pose — they record keystrokes and clicks, but nothing that says which unit of work is in progress.

The three findings that drove everything else:

1. **The case identifier is visible on screen.** 97.8% of ground-truth case IDs appear in captured screen text, 85.7% while their own execution is under way. That reframed Step 1 as case-identity reconstruction and drove the first segmenter. The delivered one separates consecutive executions of the same process with the two clicks below instead; case identity now serves the fallback and the labels (§7).
2. **Each unit of work is bracketed by two observable clicks.** Selecting a record opens it (1,780 clicks, 99.9% inside a gold execution, median relative position 0.13) and a confirm press closes it (1,752 presses, 1,751 of them inside a gold execution, never two in one, position 0.89). Using both edges lifted boundary F1 to **0.756**, and at 2-second tolerance to **0.700** — against 0.365 and 0.275 for the best baseline.
3. **One screen pattern carries 38.3% of all observed work**, in all three portal systems. That is what the automation targeted first; reading each screen's printed table header, rather than assuming it, then carried the same engine to all 12 worklist screens.

The delivered tool covers **all 12 portal worklist screens — 984 rows, 83% fully automated, zero failures**. The 17% left for a person are the rows the portal flags for judgment that no regulation its operators are seen using settles. "Fully automated" is qualified in §3: on five screens the operators consult a procedure or regulation in most runs, and there the tool performs the steps without the consultation.

The recommendation I would defend hardest is a negative one: **the LLM does not belong in the runtime path**, and the evidence for that is in §4.

1. Step 1 — recovering units of work

What the logs do and do not contain

Process mining needs an event log of *(case ID, activity, timestamp)*. The provided data has timestamps only. Recovering the other two from an uncased stream is the known problem of *event log correlation*.

What did not work, and why I checked first

The obvious approach is to segment on pauses. I measured the gap distribution before building it: **the median gap between consecutive ground-truth segments is 0.0 seconds** — half of all true boundaries have no pause at all. Built anyway as a baseline — a new segment after any pause over 3 s, the best of the thresholds from 1 to 10 s — it reached BF1@5s 0.365 and V-measure 0.062. There is no threshold at which it works: at 2 s it emits 10,928

segments for 2,009 real ones; at 10 s it finds only 11% of true boundaries within 5 seconds. A pause here is the time since the operator's last action; the agent's own gap field also counts its screenshots, and read from that the baseline scored 0.316.

A second baseline, splitting on every application switch, was also poor (V = 0.135): operators switch applications constantly *within* one unit of work.

The approach that did work

stage	signal	why
case identity	most-repeated ID in a screen capture	opening a record repeats its ID across header, fields and breadcrumb; list rows show each once. 93.5% precision
unit start	click on a table cell (`td`)	selecting the record; 1,780 clicks, 99.9% inside a gold execution, median position 0.13
unit end	click on an HTML `button`	the terminal action; 1,752 presses, 1,751 inside a gold segment (99.9%), never two in one segment, position 0.89
label	portal system + route	3 systems x 5 routes is exactly the 15 process families; V = 0.931 on gold segments

Results

Scored against dataset A's 2,009 ground-truth executions:

	best baseline	delivered (v4)	ground truth
boundary F1 @2s	0.275	**0.700**	1.000
boundary F1 @5s	0.365	**0.756**	1.000
boundary F1 @10s	0.481	**0.818**	1.000
WindowDiff (lower better)	0.682	**0.195**	0.000
V-measure (label consistency)	0.135	**0.719**	1.000
segments produced	5,236	**2,010**	2,009
idle time claimed	49.9%	**6.6%**	5.0%

One figure was never optimised for, and it is the one I trust most: the segment count lands within **0.05%** of ground truth (2,010 against 2,009), because the confirm presses fix it. Claimed idle time is within 1.5 points — 6.56% against 5.03%, which the table rounds to 6.6% and 5.0% — but that is not independent evidence: the gap expansion exists to correct idle, and its cap sets it (§7).

How honest this number is

- **The scorer was validated first.** Scoring ground truth against itself returns 1.000 on every metric. Without that check the rest is unfalsifiable.
- **Two audits found real defects.** The boundary metric originally derived boundaries from label changes, which made the boundary between two consecutive executions of the *same* process invisible — it was

scoring 94% of boundaries and silently discarding exactly the hardest 6%. A second pass found the fix had reached boundary F1 but not WindowDiff.

- **Parameter tuning leaked.** Parameters were swept over all 63 sessions with a dev/test split reported afterwards. Redone strictly, the honest protocol scored *higher* on test — the leak was real and was not inflating the result. Held out on the final pipeline: dev 0.748, **test 0.765**.
- **Generalisation was tested by holding out whole operators**, not random sessions: **BF1@5s 0.759, sd 0.117** across all eight machines, each held out in turn. Only the tuned parameter is refitted without the machine being scored; the bracketing design was developed with every machine in view. Holding out a whole department — the closer analogue of dataset B — is not possible here: every one of the 63 sessions contains work from all three. The cross-department test is dataset B itself, checked against signals the pipeline never reads (§8).
- **The evaluator was itself audited for bias**, since I chose the tolerances, the binning and the gold construction. A random segmenter with the same segment count scores BF1@5s 0.267 and ARI 0.002 — the metric discriminates by **+0.489**. Shuffling labels while keeping boundaries collapses V-measure to 0.030, so the label score is not leaking from the segmentation. And a parameter-free check agrees independently: the best-matching predicted segment covers ≥80% of a ground-truth execution **83.0%** of the time, median coverage **97.5%**, against 43.0% and 73.6% for the same segments placed at random — a control that keeps their lengths. A count-matched random segmentation, whose segments are half as long, reaches only 17.8% and 38.1%; this report used to quote that one, and it flattered the result.

Two things that audit changed. **BF1@5s has a floor near 0.26, not 0** — so the "best baseline" at 0.365 sits only 0.10 above chance and is a weaker comparator than it appeared. And it surfaced a weakness the headline was under-stating: only 35.6% of ground-truth executions overlapped exactly one predicted segment. That was a real defect, and fixing it is what produced v4: the audit showed the ends were right and the starts approximate, so the opening click was brought in as the other bracket. **The figure is now 76.8%**, with median coverage 97.5% against 74% for the same segments placed at random.

Where it fails, precisely

One held-out machine scores **0.471** against 0.72–0.86 for the other seven. It is not a tuning artefact: **LAPTOP-R36BQBTE has zero L3 events across all seven of its sessions** — the browser extension never connected, so the route signal does not exist there. A fallback that recovers the route from the L2 accessibility layer limits the damage but does not remove it.

That gives the single most useful number in this report for planning purposes: **expect BF1@5s ≈ 0.76 ± 0.12 on an unseen operator, falling to ≈ 0.47 wherever L3 capture is missing.**

The residual limit is the 23.2% of executions that still do not map cleanly to a single segment. It is not where the opening click is missing - that describes one execution in 2,009. It splits between units with both clicks (61.4% of the misses) and the 258 units with neither, which the case-anchor fallback maps to a single segment only 30.2% of the time (`explore/error_sources.py`). Screen text is captured only every ~7.7 s, so case identity between observations is interpolated rather than seen.

Deliverable: `out/segments.jsonl` — 664 segments, 15 sessions, 12 labels.

---

2. Step 2 — what the work is

Scale and people

664 executions, 173 minutes, 15 sessions, 4 operators.

Three sources disagreed on headcount and had to be reconciled: 4 `username_hash` values, 4 machine IDs (strictly 1:1), but only 3 operator names on screen. The names are **not** operators — each appears on all four machines, and each maps to one portal system at 100% consistency. They are **shared per-system logins**. So: 4 people, working across three systems under shared accounts. That last part is a governance finding, not trivia (§6).

The route names are misleading

The portal is one SPA deployed three times, so route names repeat and describe nothing. Each screen prints its own title, which names the work, and the regulation document open during it shows the kind of case most often handled:

segment label	document most often open	screen title — the work
`ops__leave-applications`	keiyaku_kaijo_tetsuzuki	契約管理 — contract management
`fin__leave-applications`	settai_keihi_kitei	経費承認（管理職） — expense approval by managers
`fin__payroll-items`	getsujitsu_teigaku_torihikisaki_ichiran	請求書承認･経費精算 — invoice approval and expense settlement
`fin__resident-tax`	shinkuitorihikisaki_touroku_tetsuzuki	発注管理 — purchase order management
`hr__onboarding`	nyusha_checklist_shinsotsu_batch	入社手続き — onboarding procedures
`ops__social-insurance`	kanrisya_kengen_shinsei_tetsuzuki	IT申請 — IT requests

This doubles as **independent validation of Step 1's labels**: the pipeline never reads document names, yet segments sharing a label consult the same document at **80.4% weighted purity**.

(A Japanese-reading reviewer verified twelve readings on 2026-09-10: the approval thresholds, the row status words, the three systems and six document names, five of them in the table above. The rest are mine and await the same check — among them the screen titles that name the processes, 新規締結 and 解除, and the notes the tool writes. Both lists are in `docs/CHECK_THIS.md`.)

Different handling within one process

The portal marks some rows for judgment: 種別 = 調整 (adjustment) on the three payroll-type screens, and 新規締結 or 解除 (a new contract, a termination) on the contract screen. Through a case ID seen inside the segment, **578 of the 664 segments (87.0%)** can be tied to the worklist row they processed, so flagged and routine rows can be compared within each process:

process	routine runs	flagged runs	median, routine → flagged	keystrokes per run	regulation open
expense_and_pay_change	93	20	11.0 → 11.5 s	4.3 → 5.3	13% → 25%
inventory_management	18	43	11.0 → 11.0 s	4.1 → 10.3	0% → 5%
invoice_approval	50	19	17.0 → 16.0 s	11.7 → 11.8	72% → 63%
contract_management	12	30	16.0 → 20.0 s	5.5 → 9.7	92% → 87%

No difference survives a correction for the 12 comparisons made (permutation tests, Bonferroni; the smallest p is 0.08, for flagged contract_management rows taking 4.0 s longer). On everything the logs record, flagged rows are handled much like routine ones: whatever judgment they need leaves no trace in time, typing or regulation reading - in a test environment whose waiting was compressed. So the portal's flag, not a measured difference in effort, is the evidence for leaving those rows to a person, and the time automation saves does not hinge on which rows are flagged. With 19–43 flagged runs per process only a large difference could have shown (`explore/handling_variants.py`).

Ranking, and why it is ranked this way

Two measurable quantities in tension:

- **Mechanical load** — clipboard events and application switches per run. What automation removes.
- **Judgment load** — share of runs with a regulation document open. What it does not.

High volume with high judgment is a poor first target however expensive it looks, because the residue is what costs the time. So the priority is the hand transfers automation would remove, discounted for judgment: executions × transfers per run × (1 − judgment share). Every process, in that order:

process	n	min	share	median	mech	judgment
expense_and_pay_change	120	27.2	15.8%	11 s	2.6	15%
inventory_management	63	15.5	9.0%	11 s	3.1	3%
purchase_orders	73	15.6	9.0%	11 s	2.1	27%
attendance_and_leave	58	13.2	7.7%	10 s	2.0	3%
invoice_approval	77	23.4	13.6%	16 s	3.9	65%
it_requests	42	8.5	4.9%	11 s	2.7	17%
benefits_applications	35	7.7	4.5%	13 s	3.0	49%
onboarding_procedures	58	19.2	11.1%	19 s	6.2	88%
contract_management	59	19.1	11.1%	19 s	6.2	88%
payments	37	11.0	6.4%	17 s	5.1	81%
manager_expense_approval	34	9.5	5.5%	16 s	4.9	79%
budget_variance_analysis	8	2.7	1.6%	21 s	3.5	0%

The order is computed before rounding; recomputed from the rounded figures shown, near-tied neighbours can change places.

A priority formula is easy to make say what you want, so the ranking was scored under **five different weightings**. Only **expense_and_pay_change** appears in the top three under all five. Two processes appear exactly once, which makes them artefacts of a particular formula rather than findings. An earlier version listed six weightings, two of them the same formula under different names, and shipped a priority that multiplied time by transfers per run — counting run length twice. It also showed this table by minutes, cut to eight processes.

The result that set the scope

Aggregating by portal **screen** instead of by system. Judgment is the share of the screen's runs with a regulation document open:

screen	executions	minutes	share	systems	judgment
payroll-items	**260**	**66.1**	**38.3%**	**3**	**27%**
leave-applications	151	41.8	24.2%	3	54%
onboarding	95	30.1	17.5%	2	85%
social-insurance	85	18.9	11.0%	3	28%
resident-tax	73	15.6	9.0%	1	27%

The two top-ranked processes are the same screen in different deployments. The HR system titles that `payroll-items` worklist 経費精算・給与変更, and the order and inventory system 在庫管理. One pattern, 38% of all work, and the lowest judgment load of the five screens — though only just, level with resident-tax and social-insurance at 27–28%. That is the target.

---

3. Step 3 — what I built

A **shared worklist automation engine** driven by one definition file per screen, covering all 12 worklist screens across the three systems.

```
tool/engine.py              the automation logic - one copy
tool/definitions/*.yaml     what differs per screen: URL, columns, regulation, wording
tool/regulations.py         extracts decision rules from the 規程 text
tool/mock_portal/           the portal, reconstructed from the logs
```

Measured result — all 984 rows across 12 screens

outcome	rows	share
routine, templated note	821	83%
flagged rows routed by regulation threshold	0	0%
fully automated	**821**	**83%**
left for a person	163	17%
failed	**0**	**0%**

Median 128 ms per row, p95 150 ms, 145 s for the full set.

Two qualifications, both measured. First, a flagged row is routed by a regulation only where the screen's operators are seen consulting it, and only if the regulation names what the row is. On this data no screen qualifies, and every flagged row goes to a person. Every flagged rule once named one expense regulation, and the approval table credited to it belonged to another: screen text is logged under the Word window in focus, which is often not the document shown. Read by its own title, the table is the entertainment-expense rules (接待交際費規程). The HR expense screen's operators never have them on screen (0 of 120 segments), and the regulation they do open (業務委託経費規程) has no approval table. So its 23 entertainment expenses go to a person with its 14 overtime-allowance adjustments, as the flagged rows of the invoice and inventory screens already did — which is why this table reads 83% where earlier versions said 93%, then 87% and 86%. Second, of the 821 automated rows, **277 are on screens whose operators open a procedure or regulation in most runs** (five screens, 65–88% of runs). There the tool performs the steps and writes 確認済 without the consultation. On the other seven screens — 544 rows, 55% of all — the recorded work is the steps themselves (`explore/automation_by_judgment.py`; R9).

The scope grew after a defect was found. The first build modelled 3 screens and 456 rows, because the fixture extractor kept only the first breadcrumb per system and hard-coded the payroll column list — so it silently found only the screens whose table matched. Reading the printed header instead of assuming it exposed **12 screens, 984 rows and 5 distinct schema archetypes**. The earlier claim that the three systems share an identical table contract was true across systems for one screen, and false across screens.

Why this process and this scope

Why this process: it is the only candidate that survived all five ranking weightings, and its screen pattern accounts for 38.3% of observed work.

Why this scope: the brief notes that breadth-versus-depth is itself the ROI question. The first build targeted this one screen, on the evidence that all three systems render it with an *identical table contract* — same seven columns, same element ids, differing only in content (`E2001` vs `BATCH-W2`, yen vs an em dash). That held across systems and not across screens, but the engine turned out not to need it: it reads each screen's printed header, so the five schema archetypes are data rather than code. One engine plus 12 definition files, none longer than 44 lines, covers every portal worklist screen; twelve bespoke scripts would repeat the same control flow twelve times.

What I deferred: the 10 of 13 regulation documents with no threshold table the tool can read — procedures, a supplier list and allowance rates with no approver. Handling them means solving procedure-following rather than rule lookup — a different and harder problem. The tool does not read them, and every row they govern that the portal flags reaches a person — as every other flagged row now does. The logs do not show which rule sets an

approver, and the client can say in one sentence what the logs cannot. Routine rows on those screens are still automated, without the consultation — the qualification R9 records.

---

4. Why this implementation form, and what I rejected

option	why not
RPA tool (UiPath, Power Automate)	Would work. Rejected on operational grounds: the client already captures DOM selectors, so a code path is more testable, diffable and reviewable than a recorded flow — and the per-system difference is data, which suits a config file rather than three recorded macros.
An LLM agent driving the UI	Measured and rejected. See below.
API integration	Preferable if it exists, but nothing in the logs evidences an API. Proposing one would be an assumption, and the brief warns against exactly that. It is among the first things to check in a real engagement; see *What I would do next*.
A script per screen	Rejected: the 12 screens share five table schemas and one control flow, so twelve scripts would repeat that flow twelve times.
Deterministic engine + config	**Chosen.**

The LLM decision, in detail

This was designed as a RAG feature: read the 規程, decide the handling. Two pieces of evidence removed it.

Measured performance. Running the judgment step through Gemini:

	deterministic	model
median latency per row	170 ms	23,310 ms (**137x**)
errors	0 / 456 (the rows then modelled)	2 / 5 (timeouts)
output quality	states the facts	echoed the regulation's *filename* back

Run again later on the delivered tool, with a key configured, it was worse. The guard refused every prompt the invoice screen produced, because that screen's 区分 carries an invoice id. The API failed on a good part of what was left. The drafts that did come back were fragments rather than sentences, and each was written over a note that was already correct. The figures are in `RESULTS.md`.

And then the reason it was never needed. The captured regulation text is a threshold table, not a judgment:

> 第 2 条（承認権限）1回あたり5万円未満：部門長承認。5万円以上：役員承認。10万円以上：社長承認。
>
> *Article 2 (Approval authority). Under ¥50,000 → department head; ¥50,000 and over → executive; ¥100,000 and over → president.*

Approval routing by amount is **arithmetic**. Arithmetic belongs in code: exact, instant, free, auditable line by line against the regulation, and incapable of inventing an approver. `regulations.py` parses those thresholds, and

`route(107158)` returns 社長承認. On a screen the regulation governs the tool writes `規程により社長承認へ回付`; none on this data is evidenced (§3), so in this run it routes no row.

The model keeps at most one job, and it would run offline: proposing a rule table from a regulation document for a human to check before it ships. The expensive, unreliable, unauditable component would run *once under supervision* rather than on every transaction. That job is designed, not built: both threshold tables in the captured regulations were parsed without a model. In the tool the model is off by default, even with a key configured; `--llm-drafts` switches on only the drafting experiment, which a later run reached again by a different route.

I would defend this as the correct answer rather than a compromise. An LLM in this path would have been slower, less reliable, more expensive, unauditable against the regulation it is supposed to implement — and would have looked more impressive.

The same question, asked of the segmenter. It rests on one hardcoded structural claim — a row click opens a unit of work, a confirm press closes it — so it is fair to ask whether a model should learn that rather than a person writing it down. `explore/learned_brackets.py` puts it to three tests: a random split; leave-one-machine-out, with the rule itself rediscovered from each training fold instead of assumed, so the comparison is not rigged in the rule's favour; and the only genuinely unseen system this project has, dataset B. The rule survives all three. A gradient-boosted detector fitted on dataset A finds a fraction of dataset B's units of work, and one blind to the page's markup fails there too, so it is not the markup carrying the result. The figures are in `RESULTS.md`. The reason is the one behind the LLM decision as well: a structural invariant of the application transfers to a department nobody tuned on, and a statistical regularity of the sessions that were observed does not.

---

5. What manual work remains, and realistic impact

Remains, by construction

- **163 of 984 rows (17%)** — every row the portal flags for confirmation that no regulation in use settles: 43 contract rows (31 new agreements, 12 terminations), 40 invoice adjustments, whose operators check a supplier list rather than an approval threshold, 37 on the HR expense screen (23 entertainment expenses, whose rules its operators are never seen opening, and 14 overtime-allowance adjustments, which no regulation names), and 43 inventory adjustments, which no regulation is seen governing — 26 of them with no amount at all. The correct boundary until the client names the rule, not a gap.
- **The other 10 of 13 regulation documents** have no threshold table the tool can read — procedures, a supplier list and allowance rates with no approver — and the tool does not read them. Where operators consult one, the tool automates the routine rows without that check; whether a person must still make it is the open question in R9.
- **Exception handling.** Nothing in the logs shows what operators do when a record is malformed or a system rejects a submission, so the tool has no designed behaviour for it beyond failing loudly.
- **Review of the automated output.** At least during rollout, a person should sample what the tool wrote.

Impact, stated carefully

The brief states the recordings were made in a test environment with compressed waiting times, so **absolute durations here cannot be converted into hours saved, and I will not do so.** What the evidence supports is relative:

- Every segment in `segments.jsonl` is portal work, and the tool now covers **all 12 portal worklist screens**.

- Across them, **83% of rows** were handled end to end without a person: 55% on screens where the recorded work is the steps themselves, and 28% on screens where operators usually consult a procedure the tool does not (§3, R9).
- So the defensible claim is that the tool addresses **the portal component of the observed workload, at 83% coverage within it** — under favourable conditions (§6), and with R9 settled screen by screen.

What it does *not* support: any statement of hours or money saved. Producing one would require production timings the client has and I do not.

---

6. Risks, and the evidence for each

Every risk below is anchored to a measurement rather than to a worry.

#	risk	evidence	mitigation
R1	**Telemetry gaps silently degrade accuracy.**	One of eight held-out machines scored BF1@5s **0.471** vs 0.72–0.86. Cause: **zero L3 events across all 7 of its sessions** — the extension never connected. Dataset B has the same gap in **1 of its 15 sessions**: 22 of its 664 segments, 6.3% of the time, come from the weaker fallback.	Monitor L3 coverage per machine as a first-class health metric; refuse to report process figures for a machine below a coverage floor. The L2 accessibility fallback limits but does not remove the damage.
R2	**The mock portal is not the real portal.**	Session handling, server-side validation, pagination, concurrency and real latency are **unobserved in the logs** and therefore unimplemented.	Treat 83% as an upper bound under favourable conditions. First engagement task: run against a staging instance before any efficiency claim is repeated.
R3	**An approach validated on one department can fail silently on another.**	Two transfers failed during this project: the anchor rule (fired 72 times in all of B) and the button naming (`btn-*-ok` matched **zero** rows in A). Each looked fine on internal statistics. A third was misjudged the other way: the case-ID prefix, 100% pure on A, was called meaningless on B, whose IDs carry a process code after all.	Never accept a transfer on internal statistics alone. Hold back one observable signal from the method and check against it — that is exactly what caught the first dataset B failure.
R4	**Automation runs under a shared account.**	Each portal system has one login shared by all four operators (100% name-to-system consistency).	No per-user audit trail exists today, so automated and human actions will be indistinguishable in the client's own logs. Needs a service account with a distinct identity before rollout, which is an access-control change, not a code change.
R5	**The regulation is the specification, and it changes.**	Where a regulation routes rows, its thresholds are hard rules parsed from document text (`5万円未満：部門長承認`), and a revised 規程 silently invalidates them. None routes today, so this risk arrives with the first screen whose regulation the client names.	Rules are extracted from the document rather than typed into code, so re-extraction is the update path. Version the rule table against the document; alert on drift; require human sign-off on each extraction.
R6	**Provider availability, if a model is ever added.**	The first live API call fell through **two 503s** before succeeding, and the default model had been retired for new keys mid-project.	Already mitigated by design: the model is off the runtime path, and the tool works with none configured.
R7	**Free-tier LLM data is used for provider training.**	Vendor terms.	No model runs unless explicitly enabled. When one is, `src/llm.py` refuses — on the only path that sends data — any prompt carrying a raw event record, a session id or a case or row id, or over 20,000 characters: enforced in code. It cannot recognise every kind of personal text, such as a name, so the one prompt the tool can send is built from a row's type, amount and classification — never its id or names. Re-run with a key configured, it refused every prompt the invoice screen produced, whose 区分 carries an invoice id.
R8	**Boundary precision is**	BF1@2s = 0.700 against 0.818 at 10 s — about 30% of true boundaries are not	Do not build anything requiring sub-5-second boundary accuracy. If needed, raise capture frequency — a collection change, not an algorithm change.

#	risk	evidence	mitigation
	structurally limited.	found within 2 s; screen text is captured every ~7.7 s.	
R9	**A templated 確認済 is not the check it names.**	On five screens operators open a procedure or regulation in most runs (65–88%), and the tool confirms 277 rows there without one.	Enable the seven low-judgment screens first. For the others, ask each screen's owners what a row is checked against, then encode it — the way a threshold table would be, once the client names the regulation — or keep a person on it; and make automated notes say they are automated, which also answers R4.

---

7. How the time was spent

A disclosure first: this was not seven calendar days. The git history runs from 9 to 12 September — four days, not a week. The brief asks how the seven days were allocated, and the honest answer is that the work was compressed. What follows is how the *effort* divided, which is the part that carries a lesson.

phase	work	why
1	Data profiling; 790 MB of JSONL → an 11 MB index	Nothing else was affordable to iterate on until this existed
2	**Evaluation harness before the segmenter**; baselines	The brief leaves "good enough" to my judgment, and that judgment is impossible without a scorer. It also killed the gap-based approach in an hour rather than a day
3	Case-ID recovery	Reframed the problem, though the final architecture barely uses it (see below)
4	v1 segmenter; LLM layer	BF1@5s 0.450. The LLM layer was premature
5	Signature labelling; **dataset B transfer failed**; two metric audits	The failure was the most valuable hour of the project
6	Terminator-driven segmenter; overfitting audit; Step 2	BF1@5s 0.450 → 0.722
7	Step 3 engine, three definitions, mock portal	94% automated, zero failures
8	Report; **evaluator bias audit**, which exposed a defect and produced v4	BF1@2s 0.286 → **0.673**
9	Continued defect hunting, as a reviewer would	Optimal boundary matching rescored everything, baselines included (BF1@2s **0.700**); Step 3 generalised from 3 screens to all 12 (984 rows); drift detection; asserted tests, each checked to fail

The allocation I would defend: roughly a third of the effort went to measurement rather than building — the harness, four audits, the held-out protocol, the transfer checks. That is a high proportion and it paid for itself repeatedly. The metric defect would have invalidated every subsequent number. The dataset B transfer failure was invisible on every internal statistic. The evaluator audit, run only because the approach was challenged, produced the single largest accuracy gain in the project.

What the final ablation says about that allocation

Removing each component and re-scoring (`explore/ablation.py`):

configuration	BF1@2s	V	ARI
shipped	0.700	0.719	0.708
without case anchors entirely	**0.723**	0.694	0.585
ends only, no opening click	0.316	0.562	0.540
labels from case prefix	0.700	0.471	0.385
one label for everything	0.700	0.011	−0.001

Two uncomfortable readings, both worth stating.

Case-ID recovery contributes nothing to boundary accuracy. Boundaries are marginally *better* without it at 2 s (0.723 against 0.700) and marginally worse at 5 s (0.750 against 0.756). It earns its place only by driving the fallback that recovers the 260 segments no confirm press brackets — which shows in ARI (0.708 vs 0.585) — and by supplying the case field used to validate dataset B. A day of work that the final architecture largely routed around.

The whole boundary gain is two clicks. Everything else — the anchors, the snapping, the tuned windows — is close to inert. That is the honest description of where the accuracy comes from, and it is also why the result is robust: `expand_gap_s` from 20 to 300 and `max_unit_s` from 100 to 600 leave BF1@2s unchanged at 0.700 — though `expand_gap_s` sets how much gap time is claimed as work, idle 12.5% at 20 s and 4.2% at 300 s against a true 5.0%, and ARI 0.658–0.720; `min_unit_s` from 1 to 10 moves it by at most 0.003, and the case-anchor threshold from 2 to 4 spans 0.695–0.714. That threshold was chosen for anchor precision, not for this score, and moving it to 4 now because it scores higher here would be tuning on the evaluation set. **There is almost nothing here that could be overfitted, because almost nothing is fitted.** The exception is the expansion cap: it was set on dataset A, and what it sets is the idle share.

Three further attempts to raise accuracy were measured, and none shipped. Each was judged against a bar written down before its full evaluation. The strongest divided the gap between units at its first app switch: boundaries improved, but label consistency fell beyond the tolerance set in advance, and one machine got worse. The delivered pipeline is unchanged, and the attempts and their numbers are in `RESULTS.md` .

The phase I would remove: the LLM layer, built well before anything needed it, on an assumption that the task wanted one. The eventual conclusion was that it should not be in the runtime path at all. That effort belonged in Step 2.

8. Honest limitations

- **Dataset B has no ground-truth file, but its row IDs carry one.** Worklist row IDs have the form P--, and the code is the process: 13 codes on 12 screens, one screen holding two. The labeller never reads it. Tied label-blind through the case anchors inside each segment, 645 of 664 segments give **V = 0.966** against the code, and 98.8% carry the label of the screen that holds it (`explore/label_vs_process_code.py`). The loss is the one merge: the HR expense screen's label holds expense claims and pay changes, which the confirmations captured after the press split 39 to 5. An anchor can name a row other than the one processed, so this estimates accuracy rather than scoring it. The portal breadcrumb, an L1 signal the

labeller cannot see, agrees at **V = 0.948** (96.8% on the system) where a breadcrumb was captured within 2 s of the segment's end, on 31 segments, against 0.158 for shuffled labels. Against the breadcrumb captured anywhere in the segment it falls to 0.480, and against the one most often in force to 0.255, because screen text is captured only every ~7.7 s and most references are stale by the time a segment ends (`explore/label_vs_breadcrumb.py`).

- **The completion-memo check no longer discriminates.** All 149 memos, which the pipeline never reads, fall inside a segment at median relative position 0.47 — but segments now cover 97.9% of session time, and 5,000 random instants give 97.9% and 0.50. Under v3, whose segments ended at the confirm press, the memos sat at 0.78; v4 extends every segment past that press. The check is reported as spent rather than quoted as evidence.
- **No measurable difference between flagged and routine rows is not proof of none.** 578 of the 664 segments tie to their worklist row (§2), but each process has only 19–43 flagged runs, so the comparison can see a large difference and not a small one.
- **Screenshots were not used.** All 34,580 screenshots that dataset A's events reference are on disk, though 3,567 (10.3%) sit in a differently named chunk folder of the same session rather than the one their event names; earlier versions of this list counted those as missing. Screen text is in the logs as text, so the images were not needed.
- **"The portal component at 83% coverage" describes 173 observed minutes on one day with four operators.** It is a measurement of this sample, not an estimate of the client's operation.

What I would do next, in order

1. Run the engine against a staging instance of the real portal (R2).
2. Check whether an API exists behind the portal. If it does, most of this tool becomes unnecessary, and that is a good outcome.
3. Instrument L3 coverage per machine as a health metric (R1).
4. Take on the 163 rows still left for a person. For the invoice, inventory and HR expense adjustments, first ask which rule sets the approver — a question for the client, not the logs. The contract rows are the procedure-following problem deferred in §3, not a matter of adding screens.